Mini Project | IoT and Applications

IoT Environmental Monitor & Warning System

How the working model senses, decides and warns

Shubhpreet Singh Verma
Anvi Jain
Eshita Agicha

Guide: Dr. Nirmal Kumar Gupta, Associate Professor

SVKM's NMIMS, School of Technology Management & Engineering, Indore | 2026–27

Numbers aren’t warnings

Manual checks

Irregular, need someone present, and give no warning in the moment a condition turns dangerous.

Single-hazard prototypes

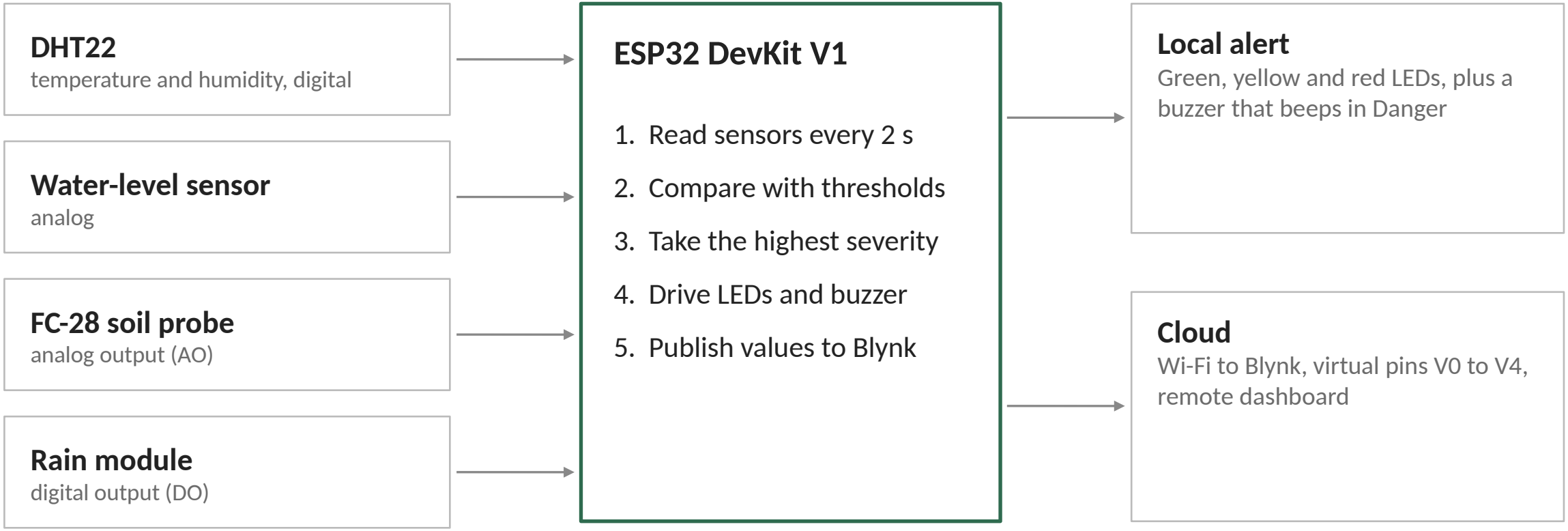
Most small systems watch one thing, flooding or irrigation, and nothing else.

Dashboard-only alerts

A phone screen is no help to someone standing at the equipment when the network is slow.

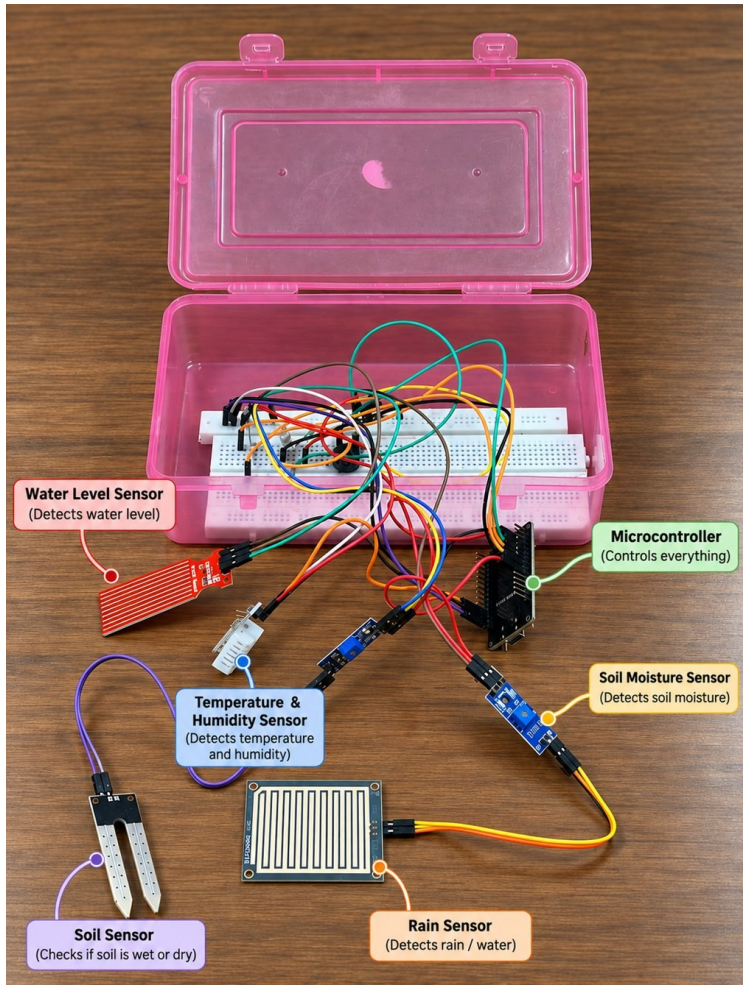
Five readings in. One status out: Normal, Warning or Danger.

Sense, decide on the device, then report



The decision is made on the ESP32. The cloud is a second channel, not a dependency.

One ESP32, four sensors, four outputs



Device	ESP32 pin	Signal
DHT22	GPIO 15	digital
Water-level sensor	GPIO 34	analog (ADC1)
FC-28 soil probe (AO)	GPIO 35	analog (ADC1)
Rain module (DO)	GPIO 27	digital
Red LED	GPIO 25	220 Ω resistor
Yellow LED	GPIO 26	220 Ω resistor
Green LED	GPIO 33	220 Ω resistor
Buzzer (passive)	GPIO 32	tone via PWM

All grounds are tied to one common ground rail. The board is powered and programmed through USB.

The four sensors at a glance

Sensor	Measures	How it works	Signal to ESP32	Pin
DHT22	Air temperature and relative humidity	A thermistor and a capacitive humidity element, read out by a small chip	Digital, single wire	GPIO 15
Water-level sensor	How much of its strip is covered by water	Water bridges parallel copper traces and lowers the resistance	Analog, 0 to 4095	GPIO 34
FC-28 soil probe	How wet the soil is	Current flows through the soil between two prongs; wetter soil conducts better	Analog AO, high = dry	GPIO 35
Rain module	Whether water drops are on the plate	Drops bridge copper traces; a comparator turns that into yes or no	Digital DO, 0 = rain	GPIO 27

The next slides take each sensor in turn: what it measures, how it works, how the ESP32 reads it, and how we use the value.

Reading signals: analog versus digital

Analog input: water and soil

- The sensor outputs a voltage between 0 and 3.3 V
- The ESP32 ADC turns it into a count from 0 to 4095 (12 bits)
- Voltage is roughly count / 4095 x 3.3 V
- We use ADC1 pins (GPIO 34, 35); ADC2 is shared with Wi-Fi
- We keep the raw count and set thresholds on it

Digital input: rain

- The pin reads only 0 (LOW) or 1 (HIGH)
- The module's comparator makes the decision; the ESP32 just reads it
- The DHT22 is also digital, but uses its own single-wire protocol handled by the DHTesp library

ADC count	**0**	**1300**	**2048**	**3500**	**4095**
Approx. voltage	0 V	1.05 V	1.65 V	2.82 V	3.30 V
Soil reading means		wet		Danger above this	dry

Counts are not physical units. Turning them into percent moisture or centimetres of water needs calibration.

DHT22: temperature and humidity

What it measures

Air temperature in degrees Celsius and relative humidity in percent, from one sensor.

How it works

A thermistor changes its resistance with temperature. A humidity element changes its capacitance as it absorbs moisture. A small chip inside turns both into digital numbers.

How the ESP32 reads it

One data wire on GPIO 15. The ESP32 sends a start pulse and the DHT22 replies with 40 bits: humidity, temperature and a checksum. The DHTesp library does this. A fresh reading is possible about every 2 seconds, which matches our timer.

In our code

```
dht.setup(DHT_PIN, DHTesp::DHT22);

TempAndHumidity th =
    dht.getTempAndHumidity();
float temperature = th.temperature;
float humidity    = th.humidity;

if (!isnan(temperature))
    Blynk.virtualWrite(V0, temperature);
```

How we use it

- Temperature: Warning from 30 °C, Danger from 35 °C
- Humidity goes to the dashboard (V1), not graded
- If a read fails (isnan): prints ERROR and skips V0 and V1

Water-level sensor: how much water?

What it measures

How much of its sensing strip is covered by water. A higher reading means more of the strip is wet.

How it works

The board has many parallel exposed copper traces. Water across the traces lowers the resistance between them, so the output voltage on the S pin rises as more traces are covered.

How the ESP32 reads it

S pin to GPIO 34, an ADC1 input-only pin. analogRead returns a count from 0 to 4095. Only the sensing strip should touch water, not the electronics.

In our code

```
#define WATER_PIN 34

int waterValue = analogRead(WATER_PIN);
Blynk.virtualWrite(V2, waterValue);

if (waterValue > 1500)
    warningLevel = 2;
else if (waterValue > 700
         && warningLevel < 1)
    warningLevel = 1;
```

How we use it

- 700 or less: Normal. 701 to 1500: Warning. Above 1500: Danger
- Dashboard shows the raw count on V2
- In our serial log the count rose 913, 1873, 1963

FC-28: how wet is the soil?

What it measures

Soil moisture. On our module the analog reading is high when the soil is dry and falls as it gets wetter.

How it works

Two metal prongs go into the soil and a small current passes between them. Wet soil conducts better, so resistance drops. The attached comparator module turns this into an analog output (AO). Its digital output (DO) is not used.

How the ESP32 reads it

AO to GPIO 35, an ADC1 input. We observed about 4095 in air or very dry soil and about 1250 to 1350 when wet.

In our code

```
#define SOIL_PIN 35

int soilValue = analogRead(SOIL_PIN);
Blynk.virtualWrite(V3, soilValue);

if (soilValue > 3500)
    warningLevel = 2;
else if (soilValue >= 2000
         && warningLevel < 1)
    warningLevel = 1;
```

How we use it

- 1999 or less: Normal. 2000 to 3500: Warning. Above 3500: Danger
- Danger here means very dry soil
- A resistive probe corrodes with use, so readings drift over time

Rain sensor: is it raining?

What it measures

Whether water drops are on its plate. It gives a yes or no, not an amount.

How it works

The plate has interleaved copper traces. A drop bridges neighbouring traces and lowers the resistance between them. A comparator on the module compares this with a level set by a small potentiometer and switches its digital output.

How the ESP32 reads it

DO to GPIO 27 as a plain digital input. 0 means rain detected, 1 means dry. The potentiometer sets how wet the plate must be to trigger.

In our code

```
#define RAIN_DO 27

int rainState = digitalRead(RAIN_DO);
Blynk.virtualWrite(V4, rainState);

if (rainState == 0) {
    // 0 = rain detected
    if (warningLevel < 1)
        warningLevel = 1;
}
```

How we use it

- Rain gives a Warning, never Danger on its own
- Serial prints RAIN DETECTED or NO RAIN
- Dashboard shows 0 or 1 on V4

LEDs and buzzer: the local alert

Three LEDs

- Red, yellow and green on GPIO 25, 26 and 33
- Each has a 220 Ω series resistor to limit current
- Current is about (3.3 − 2.0) / 220 = 5.9 mA, safe for a GPIO pin
- Exactly one LED is on at a time, matching the level

Passive buzzer

- On GPIO 32; a passive buzzer has no built-in oscillator
- It needs a changing signal, so the ESP32 PWM (LEDC) generates a 2000 Hz tone
- Danger: tone on, wait 200 ms, tone off = a short beep
- A steady HIGH would give no useful sound

Level	Green LED	Yellow LED	Red LED	Buzzer
0 Normal	ON	OFF	OFF	OFF
1 Warning	OFF	ON	OFF	OFF
2 Danger	OFF	OFF	ON	short beep

Three levels. The highest one wins

Parameter	Normal	Warning	Danger
Temperature	below 30 °C	30 to 34.99 °C	35 °C or more
Water level (ADC count)	700 or less	701 to 1500	above 1500
Soil (ADC count)	1999 or less	2000 to 3500	above 3500
Rain (digital)	no rain (1)	rain detected (0)	not a danger level

Overall level = the highest of the four.

Normal: green LED **Warning**: yellow LED **Danger**: red LED and a short beep

Humidity is shown on the dashboard only. The limits are prototype values from observed readings, not calibrated figures.

One cycle, worked through

Step	Parameter	Example reading	Grade
Read	Temperature (DHT22)	32.0 °C	Warning (30 or more)
Read	Water level (GPIO 34)	400	Normal (700 or less)
Read	Soil (GPIO 35)	2600	Warning (2000 to 3500)
Read	Rain (GPIO 27)	1 (no rain)	Normal

Overall level: Warning (highest grade)

- All LEDs are switched off, then the yellow LED is turned on; the buzzer stays silent
- The raw values are sent to Blynk on V0 to V4 and the serial monitor prints the status
- If the soil probe dried out to 4095, soil would become Danger: red LED and a short beep

Example values chosen to illustrate the logic.

A 2-second heartbeat that never waits

Every 2 seconds

1. Read DHT22, water, soil and rain
2. Check the DHT22 values with isnan()
3. Publish V0 to V4 to Blynk
4. Work out the warning level
5. Set the LEDs, beep if Danger

Built to keep running

- Starts without Wi-Fi: waits at most 20 s, then runs anyway
- Reconnects on its own: checks every 5 s, retries every 20 s
- DHTesp driver: avoids the watchdog reset we saw with the Adafruit library

From main.cpp

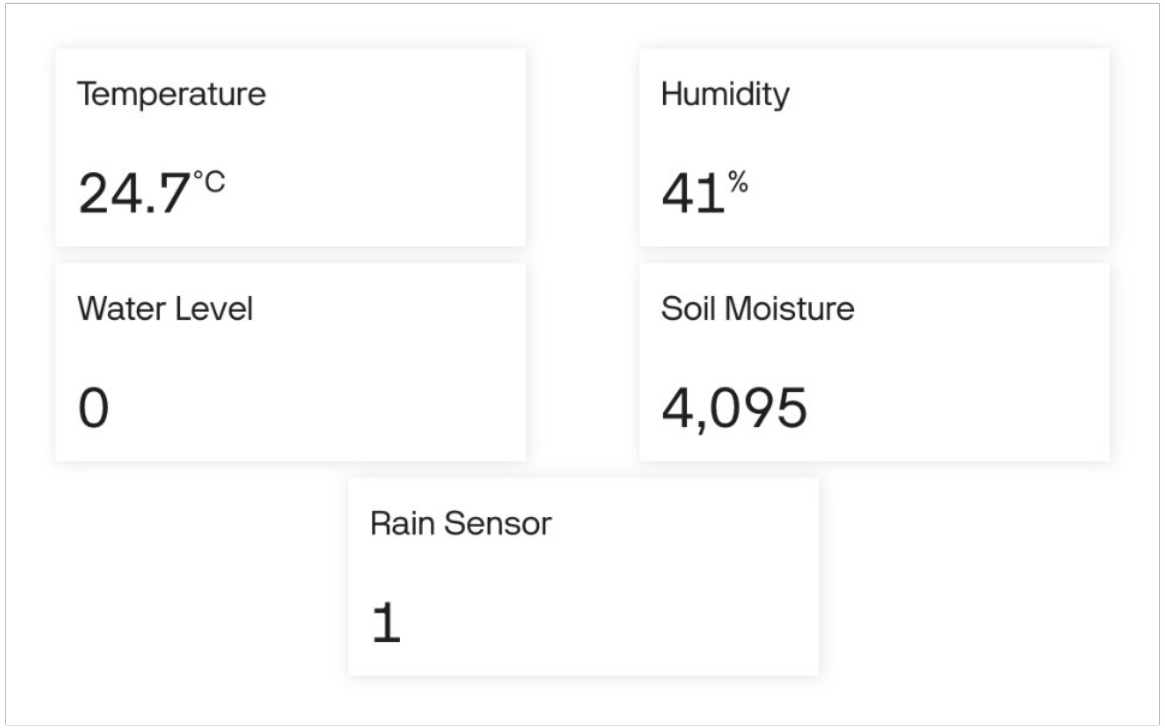
```
BlynkTimer timer;

void setup() {
  ...
  timer.setInterval(2000L, sendSensorData);
  timer.setInterval(5000L, checkConnection);
}

void loop() {
  if (Blynk.connected()) Blynk.run();
  timer.run();
}
```

Each sensor can only raise the level, never lower it.

Five datastreams, one dashboard

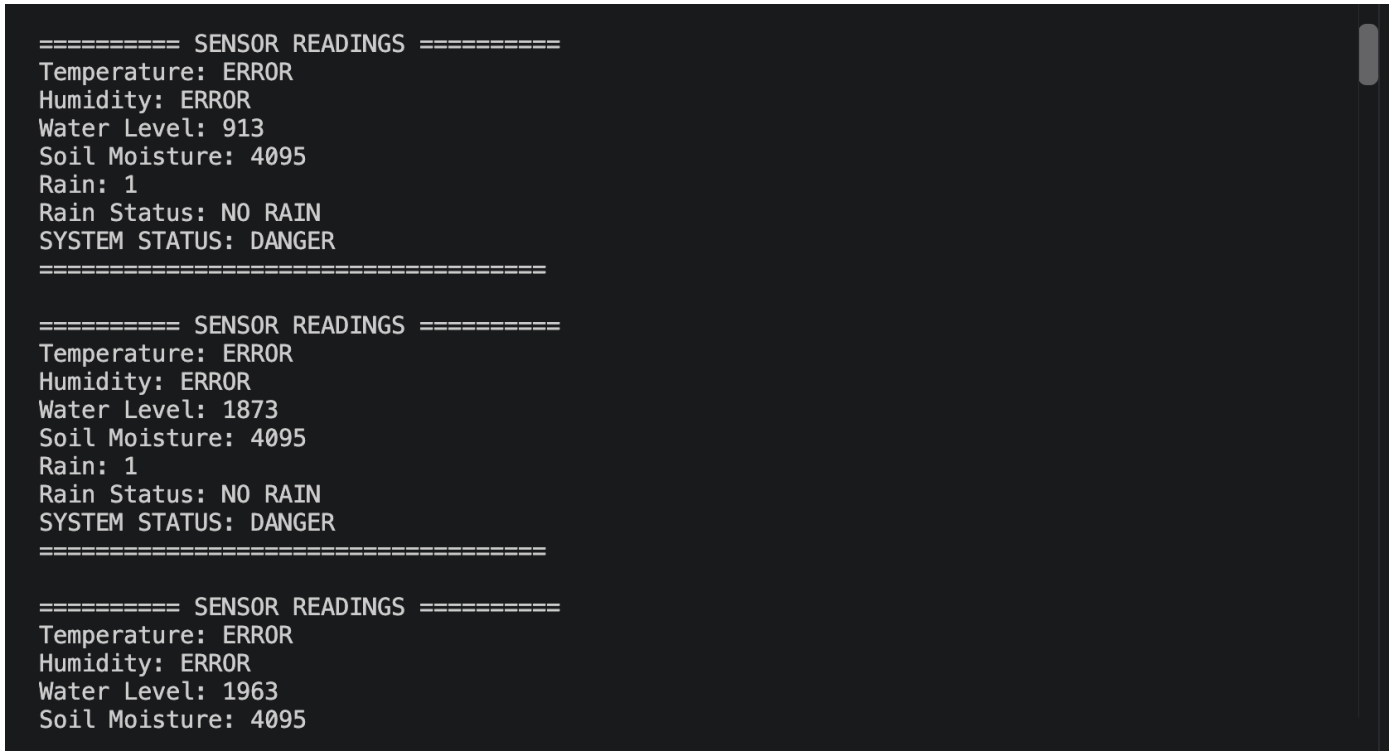


Blynk dashboard, template “IoT Environmental Monitor”

Pin	Quantity	Unit
V0	Temperature	°C
V1	Humidity	% RH
V2	Water level	raw ADC count
V3	Soil moisture	raw ADC, high = dry
V4	Rain	0 = rain detected

Virtual pins are logical channels, not GPIO pins. The firmware writes them with Blynk.virtualWrite every 2 seconds.

What the sensors actually told us



Serial log: soil 4095 gives SYSTEM STATUS: DANGER. Temperature and humidity report ERROR and are skipped.

Soil

About 4095 when dry, 1250 to 1350 when wet.

Water

Count rose 913, 1873, 1963 as it got wetter.

Rain

The module outputs 0 when rain is detected.

DHT22 fault

A failed read is flagged as ERROR and skipped, never used as data.

Honest limits, clear next steps

Limits today

- Thresholds are prototype values, not calibrated against known samples
- The FC-28 is a resistive probe: it corrodes and readings drift
- Wi-Fi password and Blynk token sit in the source file
- Breadboard build, USB powered, no enclosure

Next steps

- Calibrate with known dry and wet soil and measured water depth
- Swap in a capacitive soil probe
- Add an OLED display and Blynk push notifications
- Log the data, try anomaly detection and forecasting, then add a relay-driven pump

Thank you

Questions?

One node. Four sensors. One clear status.

Shubhpreet Singh Verma | Anvi Jain | Eshita Agicha